

区域冷链城配中心运营改善报告（PDCA）

覆盖周期：2026-06-01 ~ 2026-08-29，连续 90 天运营模拟 编制依据：本仓库 `data/processed/` 下由程序生成的结果文件。本报告每一个数字都可追溯到其中某个文件，溯源对照见文末附录 A。禁止估算、禁止把仿真/假设表述为真实观测。 配套看板：`streamlit run app.py`（运营驾驶舱）+ 仓储分析页 / 运输分析页 / 改善建议页。

摘要

维度	改善前	改善后	幅度	证据文件
拣货行走成本	3,932,220 行·米	1,538,212 行·米	-60.88%	<code>warehouse_kpi/kpi_overall.json</code>
代表日配送总里程	1,170.14 km	876.15 km	-25.12%	<code>transport/transport_kpi.json</code>
代表日用车数	11 台	10 台	-9.09%	同上
代表日柴油单均成本	39.26 元/单	34.11 元/单	-13.12%	同上
代表日纯电单均成本	35.90 元/单	32.01 元/单	-10.84%	同上

四项改善动作合计对应：**库位重排、VRPTW 路径优化、周度异常复盘机制、人力配置核查**。本报告同时如实记录**没有改善的指标**（见 C-5），以及**人力实验的结论在两次口径修正中 如何变化**（见 C-4）——第一次它消失了，第二次换了个原因回来。两次都写下来，连同「这个拐点并不是主杠杆」一起。

一、数据基础与口径声明

本工程的数据分四层，每一层的性质不同，结论的可信度也不同，报告中的每个数字都标注其所属层级：

层级	内容	性质	在本报告中的用法
真实观测	Olist 巴西电商订单 (96,282 单)、高德 POI 49 个与 50×50 真实路网矩阵	真实公开数据	作为 分布形状与异常水平 的标定基准；里程计算一律用真实路网
学术基准	Solomon 算例 C101 / R101 / RC101 + 权威 best-known 解	公认真实基准	证明优化建模能力：gap 分别为 +0.21% / -5.00% / +6.67%， 全部 ≤10% 门禁
过程仿真	仿真仓 90 天五表、SimPy 仓内作业仿真、配送订单与在途轨迹	程序生成	改善动作的 效果验证载体
情景假设	车辆成本参数、订单需求强度、在途异常倍率、库位与作业时长参数	人为设定	结论稳健性由 三档敏感性分析 承载，不依赖单一取值

必须说明的三条限制（不是免责声明，是读数的前提）：

1. 本工程是一个**教学模拟工程**，DC、车队、库位均为虚拟设定，不对应任何真实企业。
2. 情景假设参数（司机工资、油价电价、租金、货拉拉计价）的来源类别已登记，

但**具体 URL 仍待补齐**（`data_sources_ledger.md` 第 4 节标 **◆**），检索日期为 2026-09。

1. 配送侧的「时间窗达成率」与 Olist 侧的「真实准时交付率」是**两个口径**，不可互换：

前者是送达落在客户要求时间窗内的比例（配送场景），后者是实际送达早于平台预计日期的比例（巴西电商真实数据）。驾驶舱卡片显示的是前者。

二、P（Plan / 现状诊断）

以下 5 个问题全部由数据复现得出，不是主观判断。每个问题标注其数据来源层级。

P-1 拣货效率存在下午时段低谷

- **现象**：14:00–16:00 的拣货耗时为 **168.4 秒/行**，其余时段 **88.6 秒/行**，比值 **1.90**。
- **数据类别**：过程仿真（数据层 A 主动埋入的效率低谷），由 `picking_efficiency_by_hour` 独立复现。
- **证据**：`warehouse_kpi/kpi_overall.json` → `embedding_checks.pick_slowdown_14_16`；

逐小时明细 `warehouse_kpi/drilldown_picking_hour.csv`。

- **业务解读**：该低谷与仓内作业的双高峰到达（9–11 点、14–16 点）在同一下午时段叠加，

人手不变而到货集中，导致单位拣货耗时上升。

P-2 P03 品类盘点差异率显著偏高

- **现象**：P03 品类盘点差异率 **17.45%**，其余品类合计 **4.02%**，比值 **4.34**。
- **数据类别**：过程仿真（数据层 A 埋点），由分品类下钻独立复现。
- **证据**：`warehouse_kpi/kpi_overall.json` → `embedding_checks.p03_discrepancy`；

明细 `warehouse_kpi/drilldown_stocktake_category.csv`。

- **注**：盘点差异率（记录数口径）与库存准确率（金额加权口径）是**两个指标**，

前者看「多少条记录对不上」，后者看「差的钱占账面金额多少」。整体库存准确率 99.90%。

P-3 库位布局与出库频次不匹配，行走成本被浪费

- **现象**：当前库位下，出库行走成本为 **3,932,220 行·米**；折合每行拣货要走 **89.12 米**。
- **数据类别**：过程仿真（数据层 A 的「A 类 SKU 远库位」埋点）。
- **证据**：`warehouse_kpi/kpi_overall.json` → `slotting.walk_cost_before`。

- **业务解读：**A 类 SKU（141 个，占 28.2% 的 SKU）承担了 **70.0%** 的出库行数，但它们的库位距离并非按频次排序分配——高频 SKU 被放在远处，每次拣货都在为布局付费。

P-4 人工就近派车下里程与成本偏高

- **现象：**代表日（2026-07-13，132 单 / 45 个配送点）人工就近派车的最近邻贪心策略下，总里程 **1,170.14 km**、用车 **11 台**、柴油单均成本 **39.26 元**。

- **数据类别：**里程与时长来自**真实路网**（高德驾车距离矩阵），派车规则为**情景假设**。
- **证据：** `transport/transport_kpi.json` → `baseline`。
- **业务解读：**逐点就近选择是局部最优，不保证全局里程最优；车辆数也未做最小化，

导致 11 台车平均满载率仅 **70.03%**。

P-5 在途异常存在结构性偏高，且集中在特定时间与片区

- **现象**（90 天全量 9,299 单）：

靶点	高发组	对照组	比值	z	5% 显著
周五「晚点」率	7.51% (n=1,344)	3.37% (n=7,955)	2.23	+7.20	是
R07 片区异常率	15.14% (n=2,603)	5.29% (n=6,696)	2.86	+15.68	是
雨日「晚点」率	5.34% (n=3,861)	3.00% (n=5,438)	1.78	+5.69	是

- **数据类别：**过程仿真（数据层 D 埋点）；异常**总体水平**以 Olist 真实延迟率 8.13% 标定（实测生成 8.04%，差 0.09pp）。
- **证据：** `transport/anomaly_points.json`、`transport/anomaly_weekly.csv`。
- **业务解读：**周五午后拥堵叠加周末备货高峰、R07（青白江）片区路况与距离特性，

使异常在**时间**与**空间**两个维度上都不均匀。均匀的日常排班无法覆盖这种不均匀。

三、D（Do / 改善措施）

#	措施	对应问题	具体做法	实现位置
D-1	库位重排	P-3	以 $\Sigma(\text{出库频次} \times \text{库位距离})$ 为目标函数，用重排不等式（频次降序 $\leftrightarrow$ 距离升序）直接求得最优配对；库位集合不变	<code>src/warehouse_kpi.py::optimize_slotting</code>
D-2	VRPTW 路径优化	P-4	节点=POI、需求聚合（同店当日多单合并）、单点一访；载重+容积+时间窗+单 DC 往返约束；目标=固定成本+里程成本，30 秒时限；建模与 Solomon 验证同构	<code>src/transport_optimize.py::solve_vrptw</code>
D-3	周度异常复盘机制	P-5	按 ISO 周输出异常类型分布与典型案例清单，并对周五/R07/雨日三个靶点做池化与逐周的显著性检验，使「哪一周出了问题」可被当周发现	<code>src/transport_decisions.py::weekly_anomaly_summary</code>
D-4	人力配置优化	P-1	SimPy 离散事件仿真（非齐次泊松到达 $\rightarrow$ 波次批量释放 $\rightarrow$ 拣货 $\rightarrow$ 复核打包 $\rightarrow$ 发货），对 4/5/6 人各重复 30 次并报 95% CI，找时长—人力成本的拐点	<code>src/warehouse_sim.py::run_all_experiments</code>

D-2 的两处细节值得单独说明，因为它们直接决定改善是否真的成立：

- **时间窗聚合规则**：ADR-0004 原文写「同店多单的时间窗取外包络」。外包络在数学上可行但**过宽**

——求解器可以合法地把到访停在信封边缘，那一刻几乎没有任何订单真正在窗内。已收紧为「覆盖订单数最多的那一刻所属各窗的交集」，结果始终是外包络的子区间。

- **抗延误余量**：只做上面这一步时，优化方案的时间窗达成率**反而低于**朴素贪心

(15.2% vs 34.1%) ——纯成本目标把「早到等待」当纯浪费。补了 30 分钟计划缓冲 + 误点惩罚 软上界（设在**窗起**，设在窗止完全无效，因为硬约束本就禁止越界）。

四、C (Check / 效果验证)

以下每一个数字都来自程序输出文件，无一例估算。

C-1 库位重排 (对应 D-1 / P-3)

指标	重排前	重排后	变化
出库行走成本	3,932,220 行·米	1,538,212 行·米	-60.88%
每行拣货行走距离	89.12 m	34.86 m	-60.88%
每行拣货行走耗时	74.27 s	29.05 s	每行省 45.22 s

口径：以 `WALK_SPEED_M_PER_SEC = 1.2` 折算。**库位集合不变**（单测断言前后距离多重集一致），杜绝「换一批好库位刷指标」。该目标函数的最优性已用 `scipy.linear_sum_assignment` 独立复核。**交叉印证**：SimPy 实验一（独立模型）给出单均行走 -46.8%、单均拣货 -39.6%，方向一致、量级不同（两个模型不同），属互证而非替代。

C-2 配送路径优化 (对应 D-2 / P-4)

代表日 2026-07-13（132 单 / 45 个配送点），基线贪心 vs OR-Tools VRPTW：

指标	基线（人工就近派车）	优化后	变化
总里程	1,170.14 km	876.15 km	-25.12%
用车数	11 台	10 台	-9.09%
满载率均值	70.03%	77.03%	+10.00%
里程利用率	58.58%	69.10%	+17.96%
柴油总成本 / 单均	5,181.86 / 39.26 元	4,502.53 / 34.11 元	-13.12%
纯电总成本 / 单均	4,738.86 / 35.90 元	4,225.51 / 32.01 元	-10.84%

月度与年度换算（按代表日单均节省率外推，外推假设已登记台账，**不作真实财务承诺**）：

模式	日节省	月节省（×26 工作日）	年节省（×312 工作日）
柴油自购	679 元	17,663 元	211,953 元
纯电租赁	513 元	13,347 元	160,206 元

全 90 天基线覆盖（真实分母，ADR-0003）：96,932.6 km / 898 车次 / 9,299 单；其中已服务 9,037 单、**未服务 262 单**（分布于 53 天，均为送达窗在 08:00 发车后车辆可达之前 就已关闭）。全量时间窗达成率 **86.49%**。未服务单如实计入、不粉饰为「已覆盖」。

C-3 周度异常复盘（对应 D-3 / P-5）

三个靶点在周度复盘口径下**全部复现**（池化检验 z 值见 P-5 表）； 逐周复现计数：周五晚点 **6/13 周**显著、R07 异常 **13/13 周**显著。

口径说明：逐周检验的功效受单周样本量限制——周五单周样本仅 91–131 单（中位 98）。 当周不显著 **不等于**埋点不存在，故结论以池化检验为准，逐周计数回答的是「每周复盘时能否当周看出来」。两类数字都落盘（ transport/anomaly_points.json ），不把「样本不足」说成「没有问题」。

C-4 人力配置（对应 D-4 / P-1）

口径先说清楚：「**作业时长**」= **订单从波次释放到发货**（排队 + 拣货 + 复核）， 不含订单在波次窗内的累积等待——后者由作业组织决定、与人数无关，混进来只会稀释人力的效应。 两栏都给，因为端到端才是门店感知到的那个数。

拣货员	作业时长（95% CI）	端到端（到达→发货）	日人力成本	边际收益（每元换取的秒数）
4 人	487.1 s (477.9–496.3)	1,989 s	920 元	—
5 人	451.0 s (445.3–456.6)	1,956 s	1,150 元	0.1571 s/元
6 人	422.6 s (417.4–427.7)	1,937 s	1,380 元	0.1235 s/元

拐点在 5 人。三档的 95% CI 互不重叠，边际收益逐档显著下降（0.1571 → 0.1235 秒/元）。 建议**维持 5 人档**。

但这个拐点不是端到端时长的主杠杆，这一点必须一并说。 把时长按环节拆开：

环节	4 人	5 人	6 人	由什么决定
波次累积等待	1,502 s	1,505 s	1,515 s	波次窗口 ——加多少人都一样
等拣货员	283 s	218 s	166 s	人力
拣货	95 s	96 s	95 s	布局与作业方式
复核	108 s	138 s	162 s	复核台数量（3 个）

端到端 1,989 秒里 **76% 是波次累积等待**，人力碰不到它；人力能压的「等拣货员」只占 14%。 所以：

- **波次窗口是比人力大一个量级的杠杆：**窗口减半，那段等待也大致减半；
- 加人是一阶优化，且加到 6 人时瓶颈已经越过拣货侧——复核段从 108 秒涨到 162 秒，

卡在 3 个复核台上。

这一条的结论改过两次，两次都记下来。第一次（到达强度）：数据层 F 的订单到达强度取 10.0，而数据层 A 取 1.8——同一个「仓库的一天」，两层差一个数量级。10.0 会在 4 个高峰小时里堆进 87% 的订单（1.8 是 54.5%），撑得出一个明显拐点。统一到 1.8（ADR-0001 以数据层 A 为锚点）之后，拐点消失。第二次（释放环节，本次）：拐点消失的真正原因不是强度，而是仿真层缺了「释放」这一环——订单一到就抢拣货员，等于假设拣货员随时有空接单。数据层 A 一直有这一环

（`WAREHOUSE_RELEASE_DELAY_MIN = (10, 40)` 分钟，均值 25 分钟），仿真层完全没有。补上波次释放后拐点回来了，而且不再依赖任何人造参数：它是排队造成的，而排队只有在订单先被攒起来时才存在。W=0（不累积）时三档差异落在噪声里、无拐点；W≥25 分钟时拐点稳定在 5 人，且对窗口长度不敏感（25 / 50 / 100 / 200 分钟四档一致）。波次窗口 50 分钟由层 A 的释放延迟校准而来（平均累积等待 $W/2 = 25$ 分钟），见 `data/processed/sim/calibration.json` → `wave_window`。

模型边界（不可忽略）：本仿真只建模了波次窗口的等待成本，没有建模它的合并拣货收益（一个波次内多单合并成一条行走路径）。因此上面的 50 分钟不能被读作「越小越好」——真实系统里窗口变大还有省行走的一侧，本模型没有它。见 `exp2_staffing.json` → `limitations`。

C-5 两项没有改善的指标（如实记录）

1. 时间窗达成率：基线与优化同为 79.55%（实际送达口径，含在途延误）。

这不是巧合、也不是没优化：两套方案都排在窗内，延误把同样那批点推出了窗。优化的收益体现在里程与成本（-25.12% / -13.12%）而非准时。在「单点一访」规则下，该指标的理论上限已由独立计算给出：45 个配送点的当日各单窗可被一次到访全部满足，上限 100%。差距来自目标函数不含准时项——已通过延误缓冲与误点惩罚部分弥补，但未消除。

1. 在途异常总水平：实测 8.04%，与标定基准 Olist 真实延迟率 8.13% 相差 0.09pp。

这是有意为之——异常的片区/星期倍率经全样本均值归一后再抽样，差异只在时间与空间之间重分配，不抬高全局水位。全局水位若下降，说明埋点被人为放大，那是另一种失真。

五、A（Act / 固化机制）

A-1 标准化作业（SOP）

频率	动作	判据（超过即触发复盘）	数据来源
每日	查看驾驶舱四张 KPI 卡片与异常预警清单	当日异常率偏离基线（8.04%）超过 2 个百分点	驾驶舱页
每周	周度异常复盘：看本周异常类型分布与典型案例	周五晚点率或 R07 异常率当周 $z \geq 1.96$	仓储/运输页 + <code>anomaly_weekly.csv</code>
每月	ABC 分类复核与库位重排复核	A 类 SKU 的行走距离中位数上升 5% 以上	仓储分析页
每季度	车队模式与 TCO 复核	单车日均里程跌破盈亏平衡点 46.3 km	运输分析页
触发式	人力档位复核	平均履约时长上升 10% 或拣货员利用率持续 >85%	仓储分析页

A-2 看板常态化监控

四张页面即日常监控入口，全部读数直接来自 `data/processed/` 产物，**不硬编码**，因此看板与报告不会出现两套数字：

- **运营驾驶舱**（`app.py`）：库存准确率、时间窗达成率、满载率、单均成本四张带环比的卡片，近 30 天趋势，最新异常预警清单。支持全局筛选（日期范围 / 品类 / 配送区域）。
- **仓储分析页**：ABC 帕累托、分时段拣货效率（可见 14–16 点低谷）、800 库位频次热力图、盘点差异 Top10 品类、Olist 履约时长分布与州下钻、SimPy 对照实验与人力滑块。
- **运输分析页**：Folium 真实地图切换优化前/后路线（含各点时间窗）、前后对比、满载率分布、异常类型与周趋势、温控达标率仪表、双模式 TCO 曲线与外包对照。
- **改善建议页**：本报告 P→D→C 的可交互版本，what-if 控件全部读预计算缓存、瞬时响应。

A-3 机制层面的固化

1. **口径单一来源**：所有成本/里程/KPI 公式只在 `src/config.py` 与各模块的纯函数里实现一次，看板层不再重写。已发生过的两次口径分叉（概率模型复制成两份、成本公式重写）均已成文记录。

1. **数字可追溯**：每个数字的取值、数据类别、来源与口径见 `data_sources_ledger.md`；台账中未补齐的项一律显式标注 **◆**，不伪装已溯源。

1. **失败不伪装**：脚本缺输入直接报错并给出运行顺序；看板缺产物列出该跑的命令而不是显示 0；PDF 生成不了就只交付 Markdown 并写明手动转换步骤（见下节）。三条都写进了测试或断言。

附录 A：数字溯源对照表

报告中的数字	产物文件	生成命令
拣货 168.4 vs 88.6 秒/行、P03 盘差 17.45% vs 4.02%、库存准确率 99.90%、收货及时率 81.49%、拣货 33.1 行/人时、履约 0.99 h、库容利用率 62.50%、盘差率 5.29%	data/processed/warehouse_kpi/kpi_overall.json	python -m src.warehouse_kpi
行走成本 3,932,220 → 1,538,212 行-米、每行 89.12 → 34.86 m	同上 → slotting	同上
ABC: A 类 141 个 SKU 占 70.0% 出库行数	同上 → abc	同上
代表日 1,170.14 → 876.15 km、11 → 10 台、满载率 70.03% → 77.03%、成本降幅	data/processed/transport/transport_kpi.json 、 baseline_vs_optimized.csv	python -m src.transport_optimize
全 90 天 96,932.6 km / 898 车次 / 9,037 已服务 / 262 未服务 / 86.49%	同上 → baseline_all_days	同上
月/年节省金额、双模式 TCO、盈亏平衡 46.3 km、外包对照	data/processed/transport/transport_tco.json	python -m src.transport_decisions
周五 7.51% vs 3.37% (z=+7.20) 、R07 15.14% vs 5.29% (z=+15.68) 、雨日 5.34% vs 3.00% (z=+5.69)	data/processed/transport/anomaly_points.json	同上
逐周异常率与案例清单	data/processed/transport/anomaly_weekly.csv 、 anomaly_cases.csv	同上

报告中的数字	产物文件	生成命令
Olist: 96,282 单、 准时率 91.87%、延迟率 8.13%、履约 12.57 天	data/processed/olist_kpi/overall_kpi.json	python -m src.olist_kpi
Solomon gap: +0.21% / -5.00% / +6.67%	data/processed/solomon/solomon_validation.json	python -m src.solomon_validate
人力拐点 5 人 (作业 487.1 / 451.0 / 422.6 s, 边际 0.1571 → 0.1235 s/ 元)	data/processed/sim/exp2_staffing.json	python -m src.warehouse_sim
端到端时长分解: 波 次等待 1,502 / 等拣 货员 283 / 拣货 95 / 复核 108 秒 (4 人 档); 波次窗口敏感 性	同上 → arms / wave_sensitivity	同上
波次窗口 50 分钟的 校准依据 (层 A 释放 延迟均值 25 分钟)	data/processed/sim/calibration.json → wave_window	同上
实验一单均行走 -46.8%、单均拣货 -39.6%	data/processed/sim/exp1_layout.json	同上
温控达标率、总体异 常率 8.04%、车辆低 满载	data/processed/qc/delivery_qc.json	python -m src.gen_delivery_data

附录 B：报告复现

```
python -m src.gen_warehouse_data      # 数据层 A
python -m src.olist_clean              # 数据层 B
python -m src.geo_poi && python -m src.geo_matrix  # 数据层 C (无密钥走缓存)
python -m src.gen_delivery_data       # 数据层 D
python -m src.solomon_validate        # 数据层 E (gap>10% 不放行)
python -m src.warehouse_sim           # 数据层 F
python -m src.olist_kpi && python -m src.warehouse_kpi  # 模块一
python -m src.transport_optimize && python -m src.transport_decisions  # 模块二
python report/build_pdf.py            # 由本 Markdown 生成 PDF
```

PDF 交付说明：本报告提供 Markdown 与 PDF 两版。PDF 由 `report/build_pdf.py` 调用本机 headless Edge（或 Chrome）打印生成，带中文字体 CSS。若运行环境检测不到 Edge/Chrome，脚本会**降级为只交付 Markdown**，并在运行日志与本 README 中显式告知「PDF 未生成、原因、手动转换步骤」，**不伪装已生成**。

本报告由程序生成的结果文件编制，不含任何人工估算数字。所有情景假设参数的取值与来源见 `data_sources_ledger.md`，领域术语定义见 `CONTEXT.md`，架构决策见 `docs/adr/`。